

page. After successful decryption, the next step is to decompress the parameter values and create a new *IState* object from the obtained bytes. Here, after getting the *IState* object, HDIV will now check each of the parameters and the values contained in them. Validation on relative values is also done. If the request is correct, it is redirected to the Struts controller to generate the corresponding page. Otherwise, if a security error is detected, the user is redirected to an error page and the incident is logged. All the above steps can be seen in Figure 1.

Hash strategy

Unlike the cipher strategy, the hash strategy only guarantees integrity of data and not confidentiality. The main difference between these is that instead of creating a cipher, a hash is created of the *IState*, which is stored in the user session at the server end.

The steps followed in the hash strategy are as follows:

HDIV collects all request data and generates an *org.hdiv.state.IState* object, encodes it in Base64, and sends it to the client in a hidden form field or link parameter, *_HDIV_STATE*. A hash of the *IState* value is also created and stored in the user session at the server side. Now, while validating the request, HDIV computes a hash of the *_HDIV_STATE* value, and compares it with the hash value stored in the user session. If the hashes match, it is redirected to the Struts controller to generate the corresponding page, otherwise, to an error page. For a pictorial representation of these steps, see Figure 2.

Memory strategy

In this strategy, the state of each request is stored in the user session at the server end only, without any encryption or encoding, and is not sent to the client. Only a request identifier is sent to the client, which can later be used to recover the request state. Before initialising validation, HDIV obtains the request identifier (and thus, the *IState* object) from the user session. From here on, validation is the same as the above strategies. The memory strategy also provides both data integrity and confidentiality.

Installation and configuration

HDIV adds security functionalities to different versions of Struts 1.x, Struts 2.x, Spring MVC, Jakarta Taglibs (JSTL) and JavaServer faces by extending its behaviour and maintaining the API and the specification. Different HDIV versions support different versions of development platforms, and hence installation also differs from platform to platform (please refer to the links given at the end of the article). We'll discuss installing HDIV for Struts 2.x.

HDIV installation and configuration is based upon four files:

- *hdiv-config.xml* : HDIV configuration file.
- *pplicationContext.xml* : Contains bean definitions.
- *struts-plugin.xml* : Struts base configuration file, which contains new classes to use by the Struts core, and an interceptor for the validation of editable data. Interceptor

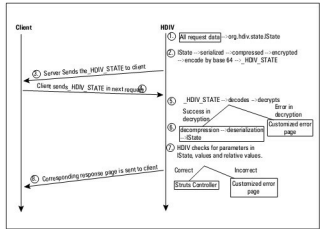


Figure 1: Cipher strategy

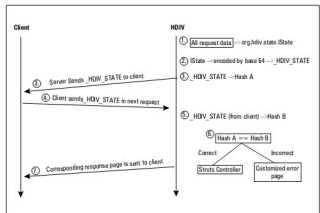


Figure 2: Hash strategy

is used to be able to see the errors generated by HDIV's generic validation for editable data.

- *hdiv-validations.xml*: Contains configuration for editable data validations.
- Installing HDIV is a three-step process.

Installing libraries

Include the following libraries in the Web application CLASSPATH, i.e., WEB-INF/lib or in server level CLASSPATH:

- Core library (*hdiv-core-2.0.x.jar*)
- Tags library (*hdiv-struts-2.0.x.jar*).

The deployment descriptor for the Tags library is *hdiv-tags.tld*, which extends the behaviour and syntax of Struts Generics Tags and HTML tags in order to achieve data integrity and confidentiality. To use the tags from this library, add the following directive to the top of your JSP page:

```
<%@ taglib prefix="form" uri="http://www.hdiv.org/struts2/tags" %>
```

Here, *form* is the tag name prefix to be used in order to add a new hidden field containing *_HDIV_STATE*, which will be sent to the client.